Lab Report: Basic Packet Capture and Protocol Analysis with Wireshark

Student Name: Suraj Somkuwar

Date: 21st Feb 2026

Environment: Kali Linux (VMware Workstation)

Objective

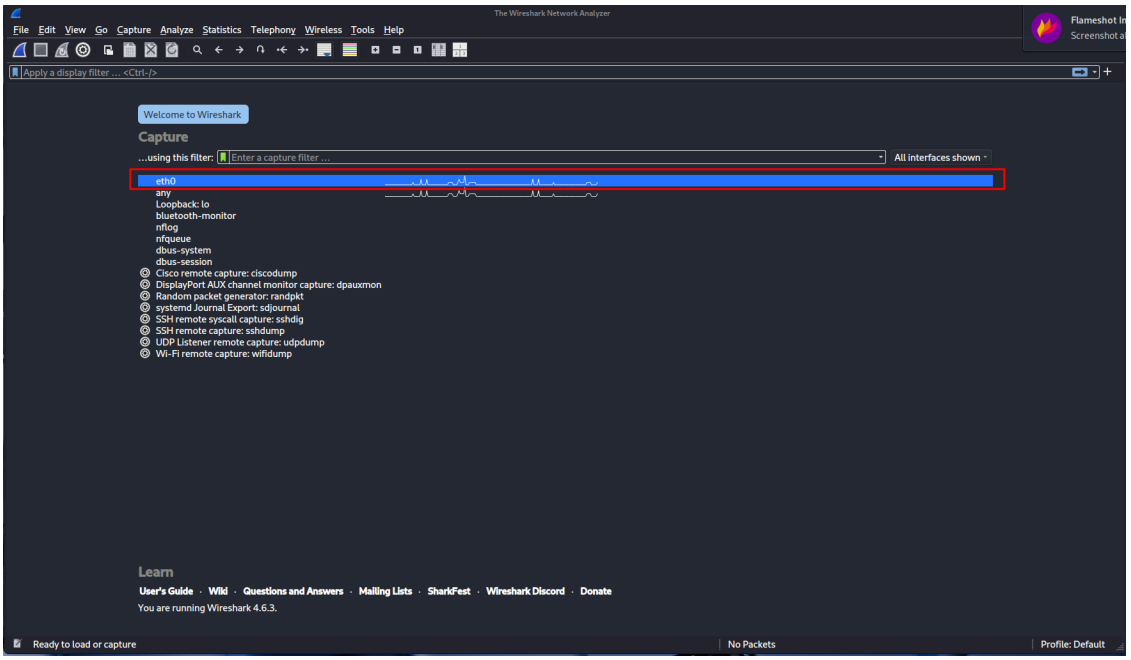
To learn the basics of packet capture and analyze standard network protocols (TCP, DNS, HTTP) using Wireshark.

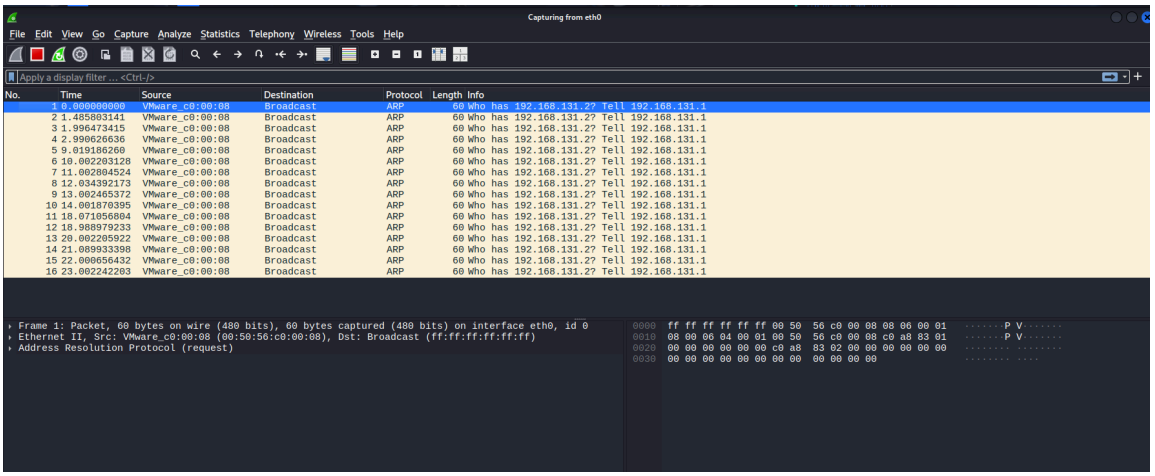
Procedures and Observations

Launch and Interface Selection

1. **Launched** Wireshark within the Kali Linux environment.
2. **Identified and selected** the `eth0` network interface. The live, increasing packet counters on `eth0` confirmed active network traffic.

- **Observation:** The selection of the active interface was successful, indicated by the live packet count.

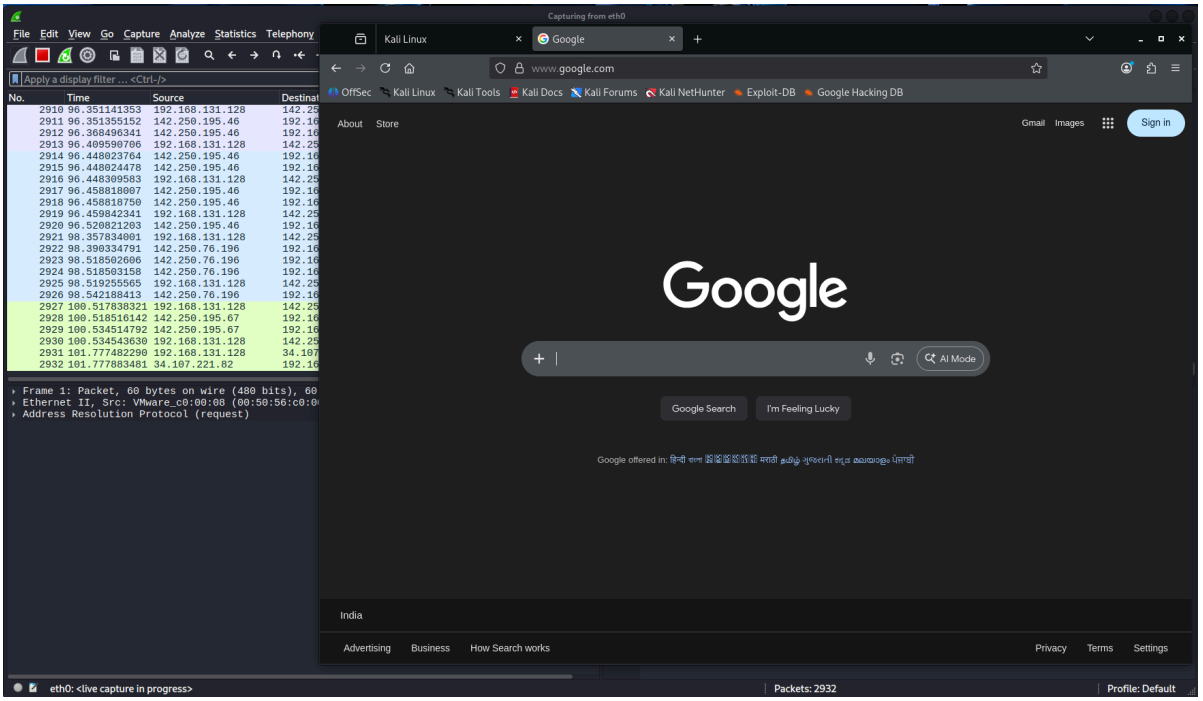


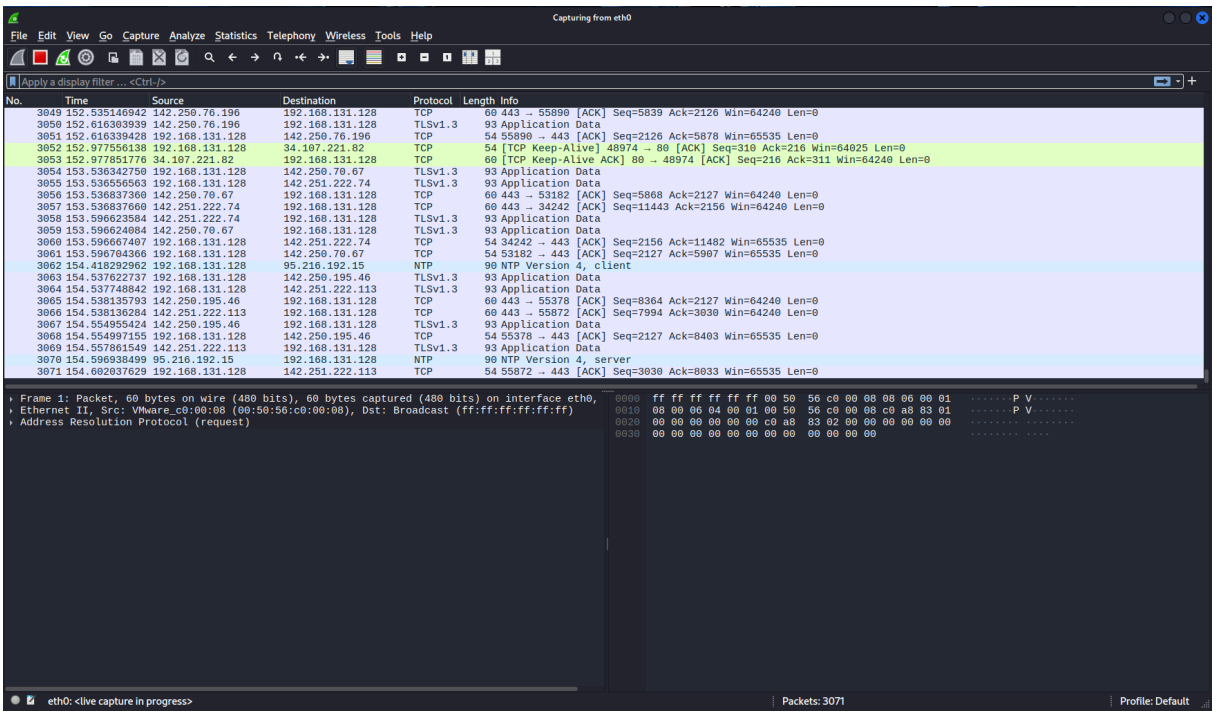


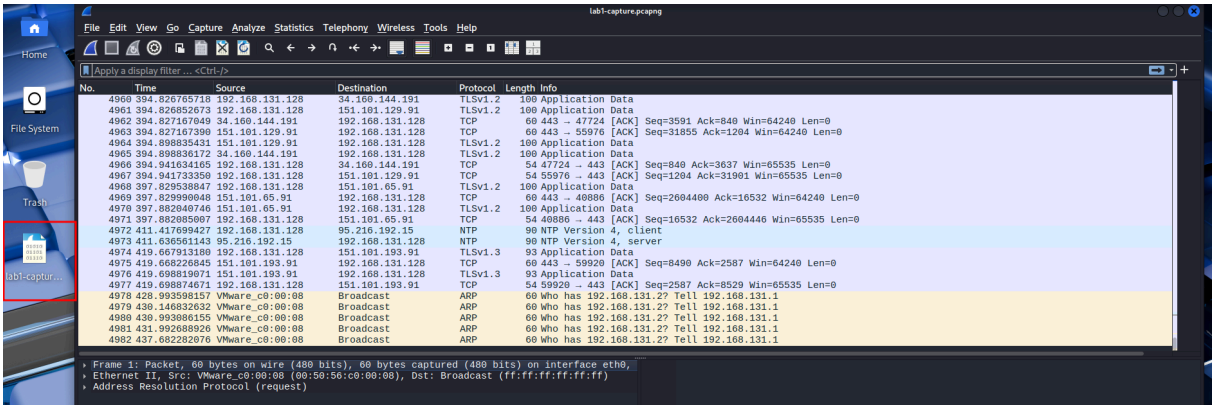
Capturing and Generating Traffic

1. **Initiated** the packet capture on the `eth0` interface.
2. **Generated** network traffic by opening a web browser and navigating to `(https://www.google.com)`.
3. **Stopped** the capture and **saved** the data as `lab1-capture.pcapng`.

- **Observation:** A diverse set of packets were successfully captured during the browsing session.



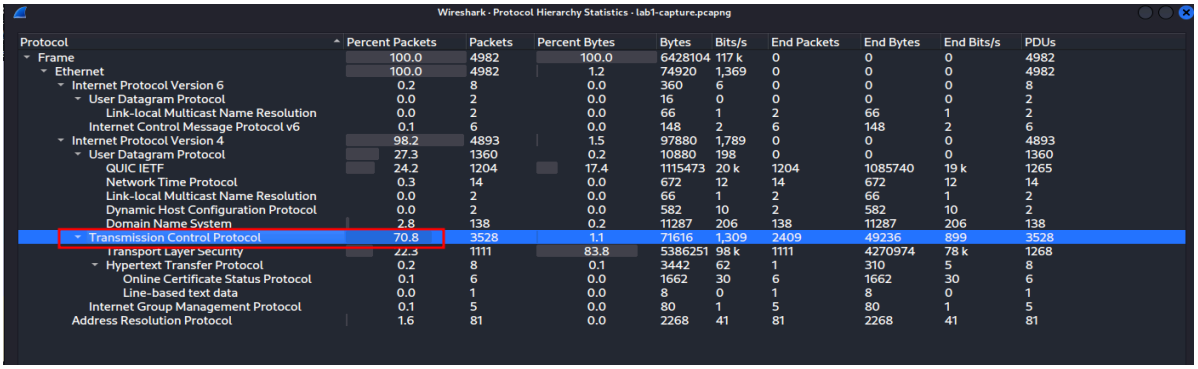




Analyze Protocol Distribution

1. **Navigated** to **Statistics > Protocol Hierarchy** to view the distribution of captured protocols.
2. **Identified** Transmission Control Protocol (TCP) as the dominant protocol under IPv4, accounting for **70.8%** of the total packet traffic.

- **Observation:** TCP is the most prevalent protocol, consistent with web browsing activity.

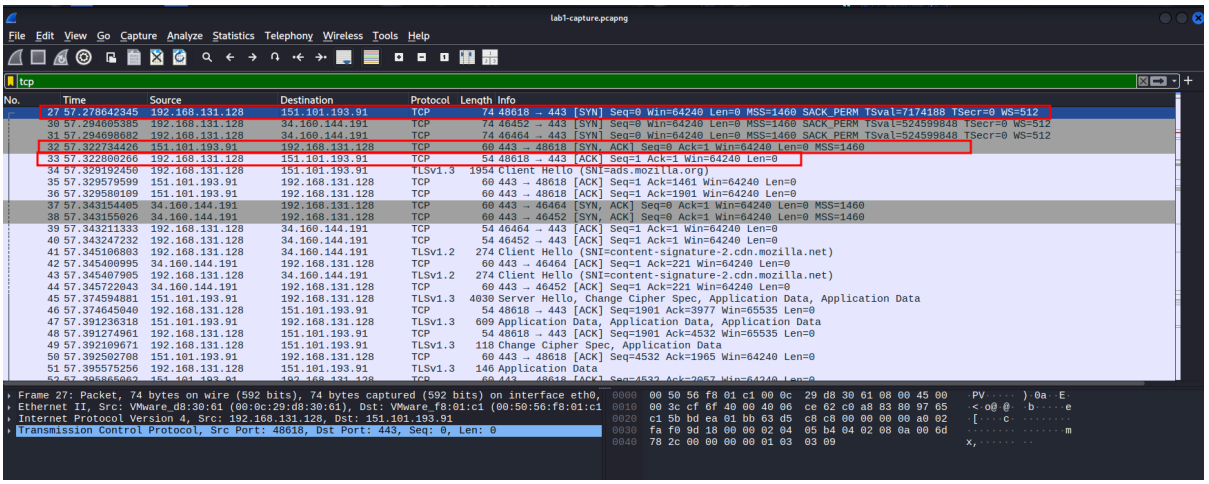


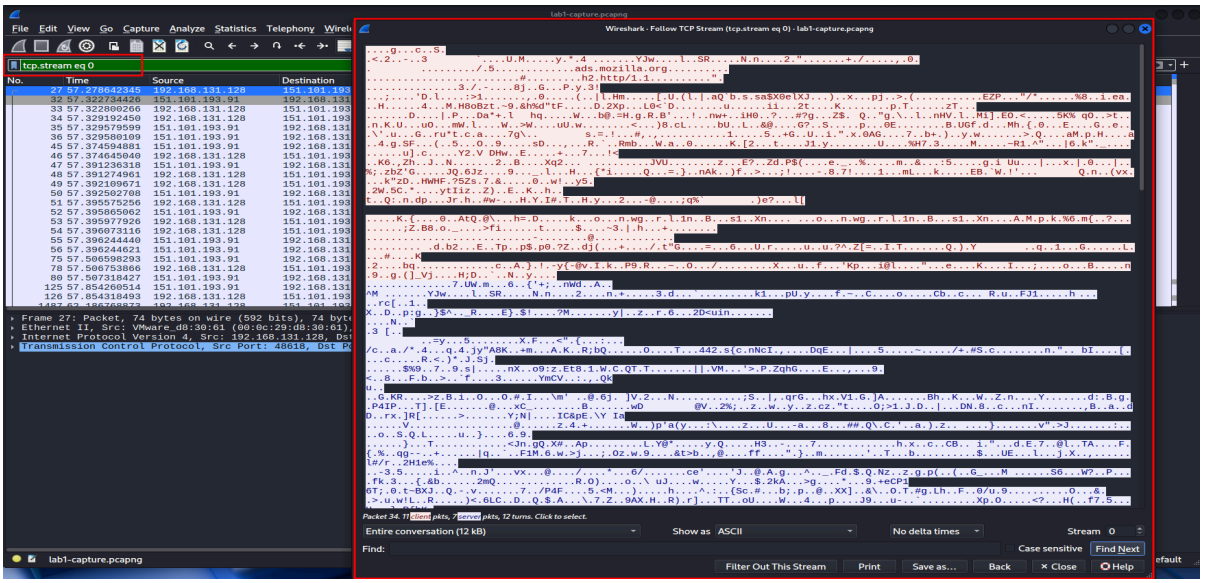
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	4982	100.0	6428104	117 k	0	0	0	4982
Ethernet	100.0	4982	1.2	74920	1,369	0	0	0	4982
Internet Protocol Version 6	0.2	8	0.0	360	6	0	0	0	8
User Datagram Protocol	0.0	2	0.0	16	0	0	0	0	2
Link-local Multicast Name Resolution	0.0	2	0.0	66	1	2	66	1	2
Internet Control Message Protocol v6	0.1	6	0.0	148	2	6	148	2	6
Internet Protocol Version 4	98.2	4893	1.5	97880	1,789	0	0	0	4893
User Datagram Protocol	27.3	1360	0.2	10880	198	0	0	0	1360
QUIC IETF	24.2	1204	17.4	1115473	20 k	1204	1085740	19 k	1265
Network Time Protocol	0.3	14	0.0	672	12	14	672	12	14
Link-local Multicast Name Resolution	0.0	2	0.0	66	1	2	66	1	2
Dynamic Host Configuration Protocol	0.0	2	0.0	582	10	2	582	10	2
Domain Name System	2.8	138	0.2	11287	206	138	11287	206	138
Transmission Control Protocol	70.8	3528	1.1	71616	1,309	2409	49236	899	3528
Transport Layer Security	22.3	1111	83.8	5386251	98 k	1111	4270974	78 k	1268
Hypertext Transfer Protocol	0.2	8	0.1	3442	62	1	310	5	8
Online Certificate Status Protocol	0.1	6	0.0	1662	30	6	1662	30	6
Line-based text data	0.0	1	0.0	8	0	1	8	0	1
Internet Group Management Protocol	0.1	5	0.0	80	1	5	80	1	5
Address Resolution Protocol	1.6	81	0.0	2268	41	81	2268	41	81

Analyze TCP Handshake

1. **Applied** the `tcp` display filter to isolate TCP traffic.
2. **Successfully identified** a complete three-way handshake sequence (**SYN, SYN-ACK, ACK**) between the Kali VM and a destination server.
3. **Utilized** the "**Follow > TCP Stream**" feature to isolate the conversation (`tcp.stream eq 0`) and observed the encrypted TLS data payload.

- **Observation:** The foundational TCP connection establishment process (3-way handshake) was verified. The data payload was encrypted, as expected with HTTPS.

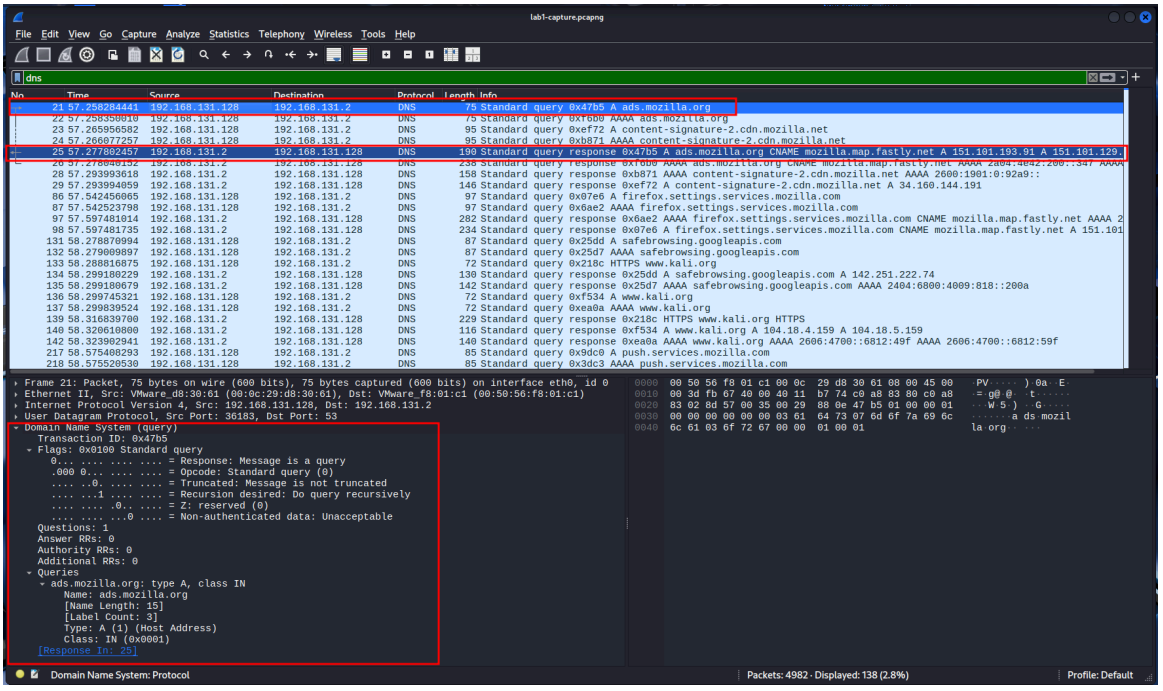




Analyze DNS Queries

1. **Applied** the `dns` display filter.
2. **Examined** a DNS query packet requesting the **A record** for the domain `ads.mozilla.org`.
3. **Located** the corresponding DNS response packet, which successfully resolved the domain name to the IP address **151.101.193.91**.

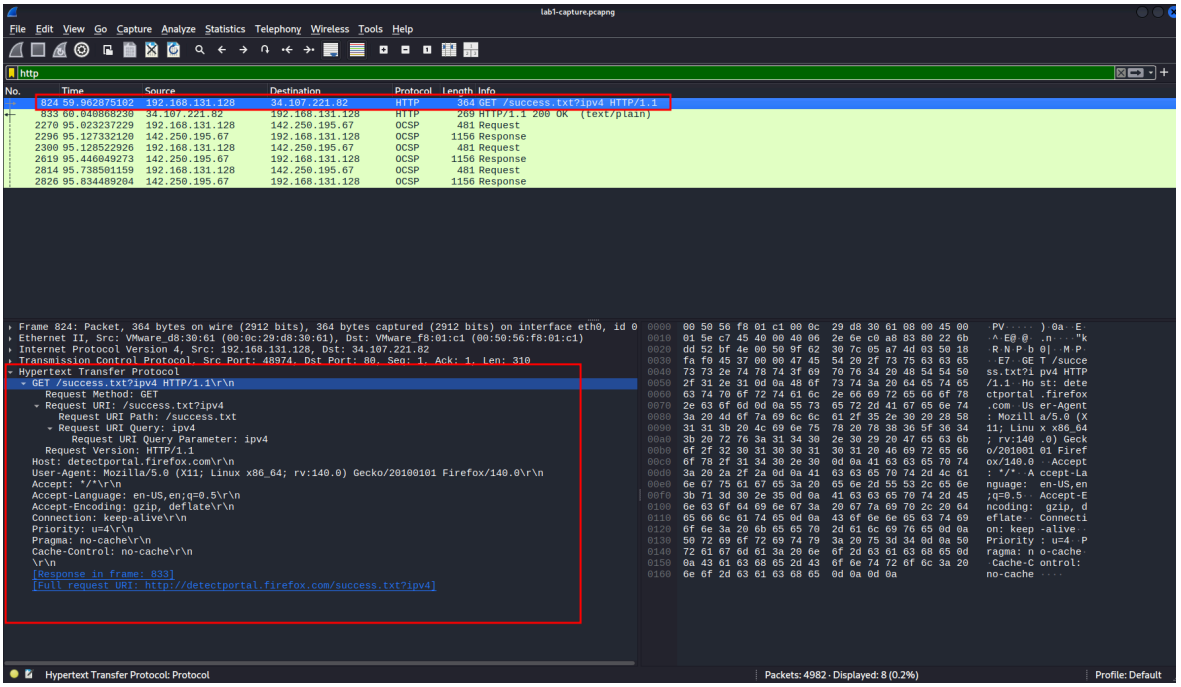
- **Observation:** DNS name resolution was successfully captured, showing the cleartext query and the resulting IP address mapping.

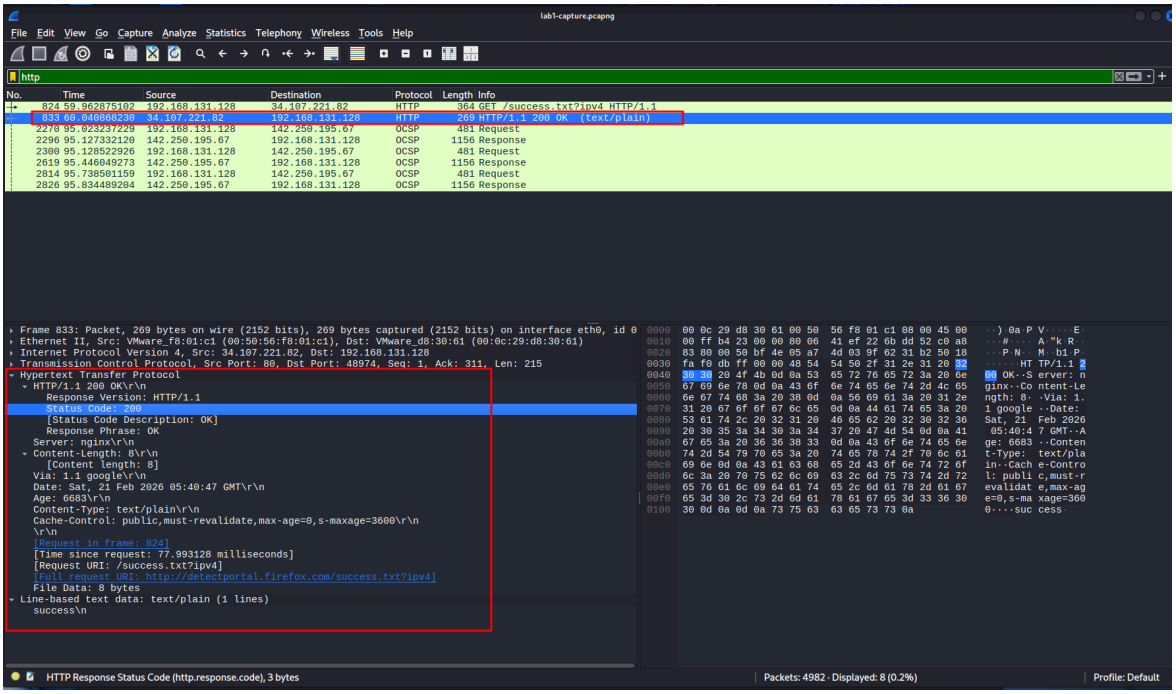


Analyze HTTP Traffic

1. **Applied** the `http` display filter.
2. **Identified** an HTTP **GET request** and **expanded** the Hypertext Transfer Protocol pane to view headers, including **Host** and **User-Agent**.
3. **Examined** the subsequent response packet and verified a status code of **200 OK**, indicating a successful request.

- **Observation:** The cleartext nature of the HTTP protocol was confirmed, allowing easy inspection of request headers and the successful response status.





Conclusion

This lab successfully demonstrated the foundational ability to capture live network traffic on a virtualized Kali Linux interface using Wireshark. By utilizing display filters, I was able to isolate and analyze specific protocols, verify the TCP 3-way handshake mechanism, observe the process of DNS name resolution, and inspect cleartext HTTP requests and responses. The exercise provided practical experience in network traffic analysis, confirming the theoretical understanding of these core internet protocols.